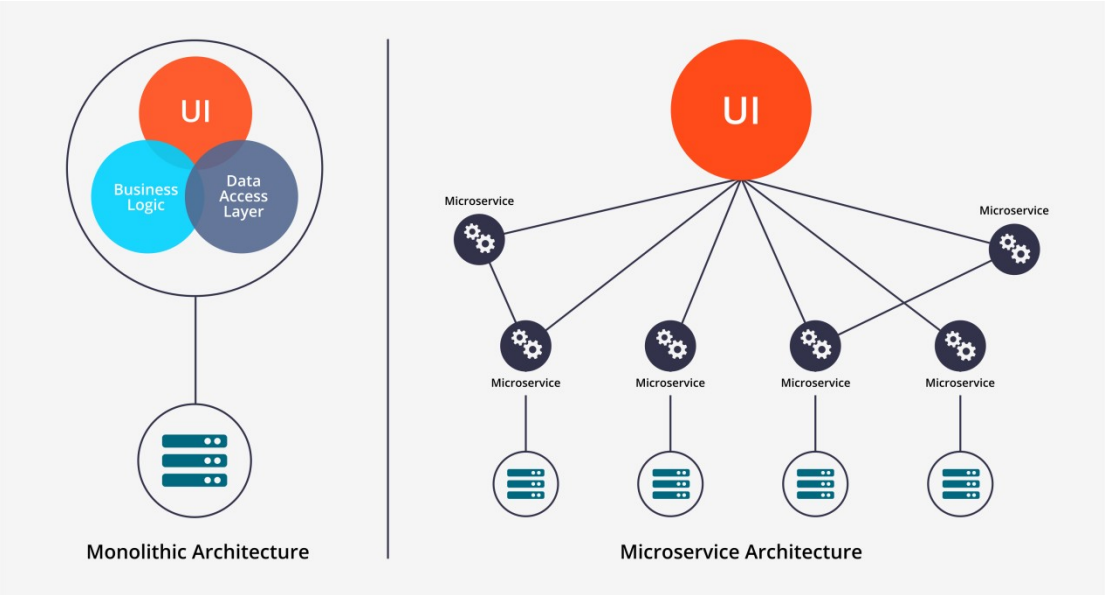
Mikroservisi

1. Uvod

Implementacija softverskog rešenja kroz monolitnu aplikaciju može biti odgovarajući pristup na početku životnog veka aplikacije, dok je broj podržanih funkcionalnosti mali i razvoj, testiranje i deployment su brzi i jednostavni. Međutim, kako vremenom broj novih zahteva raste, aplikacija postaje sve kompleksnija, samim tim teža za razumevanje, održavanje i dalja proširenja. Prednosti koje su bile vidljive na samom startu počinju da nestaju. Osnovni razlog je taj da postoji jedan codebase, deljen među svim timovima koji učestvuju na projektu, što otežava usklađivanje izmena koje dolaze sa raznih strana i agilan razvoj postaje sve teži. Počinju da se javljaju problemi kao što su onemogućen continuous deployment, neusklađeni zahtevi za skaliranjem različitih modula, preveliki rizik pri izmeni tehnološkog steka jer se cela aplikacija mora prepisati itd.

Jedno od rešenja za aplikacije koje su prerasle monolitnu arhitekturu jeste mikroservisna arhitektura. Mikroservisi predstavljaju aplikaciju implementiranu kroz kolekciju servisa koji su slabo povezani, jednostavni su za održavanje i testiranje, njihov deployment je međusobno nezavisan, organizovani su oko poslovnih zahteva i za njih je zadužen mali tim. Svaki mikroservis svoju unutrašnju logiku enkapsulira u API koji je vidljiv ostalim mikroservisima i sva komunikacija obavlja se preko mreže. Razlika između monolitne i mikroservisne arhitekture prikazana je na slici ispod.



Jedno od osnovnih pitanja koje se postavlja pri razvoju mikroservisa jeste kako aplikaciju podeliti na mikroservise i koliko mali ti servisi treba da budu. Mikroservis treba da bude zadužen za jednu poslovnu celinu koja predstavlja

deo čitavog sistema, odnosno treba da poštuje Single Responsibility Principle. Međutim, gde se tačno nalazi granica između odgovornosti različitih servisa? Zlatno pravilo ne postoji, ali dobre smernice za tu podelu nudi Domain-Driven Design i koncept koji on uvodi, a to je bounded context. Kompleksni domenski modeli mogu se segmentisati na bounded kontekste, odnosno poglede na isti domen iz različitih perspektiva. Dalje, identifikovani bounded konteksti mogu se mapirati na mikroservise.

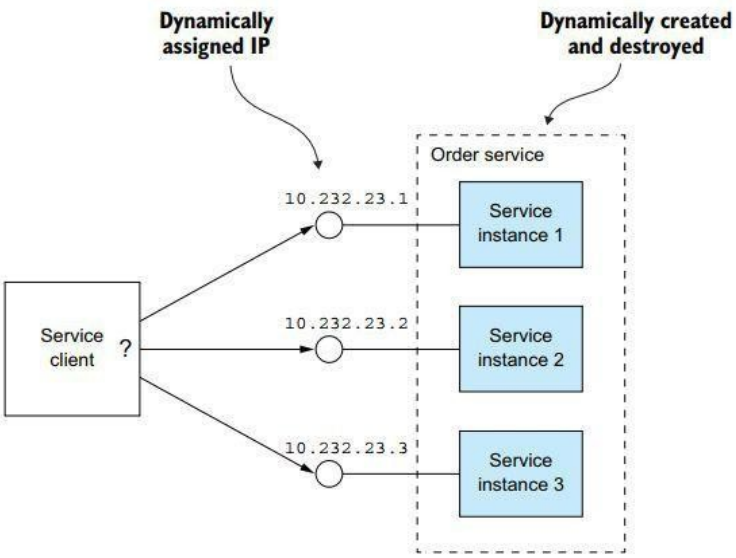
Za kompleksne aplikacije koje imaju tendenciju da rastu i razvijaju se mikroservisna arhitektura predstavlja pogodno rešenje. Neke od prednosti koje ova arhitektura donosi su sledeće:

- Poboljšana modularnosti i smanjena kompleksnost jer svaki mikroservis poseduje svoj codebase
- Razvoj novih funkcionalnosti bez ili sa minimalnim uticajem na ostatak sistema
- Omogućen continuous delivery i deployment
- Nezavisan deployment pojedinačnih servisa
- Ovakva arhitektura je visoko skalabilna
- Jednostavno uvođenje upotrebe novih tehnologija

Mikroservisi nisu nužno pogodno rešenje za svaki problem i dolaze sa svojim manama i izazovima. Podela sistema na mikroservise predstavlja složen posao i ukoliko nije ispravno obavljen prednosti koje ovakva implementacija donosi lako mogu biti izgubljene. Takođe, mikroservisni sistem je po svojoj prirodi distribuiran što uvodi dodatnu kompleksnost pri komunikaciji, kao i potrebu da se nosimo sa situacijama kada delovi sistema otkažu, nisu dostupni ili izazivaju kašnjenje. Transakcije ili upiti koji se protežu kroz više servisa moraju se pažjivo koordinisati.

2. Service Discovery

Za razliku od monolitne arhitekture gde je skaliranje moguće samo na nivou celog sistema pokretanjem većeg broja instanci, kod mikroservisa su mogućnosti skaliranja mnogo šire i ono se vrši granularno, prema trenutnim zahtevima. Broj pokrenutih servisa može biti izuzetno velik i dodavanje i uklanjanje instanci je dinamično, što naglašava problem određivanja lokacije servisa. Primera radi, ako želimo da pošaljemo zahtev nekom REST servisu, moramo da znamo njegovu IP adresu i port. Ako imamo 5 mikroservisa, pri čemu je svaki pokrenut u 5 instanci, moramo znati 25 adresa. Prvi način koji je uglavnom moguć, ali zbog dinamike i prirode arhitekture nije poželjan jeste da imamo neki konfiguracioni fajl u kome će biti zapisane konkretne lokacije i koji ćemo periodično održavati. Zbog kompleksnosti današnjih sistema, održavati ovako nešto je jako teško.

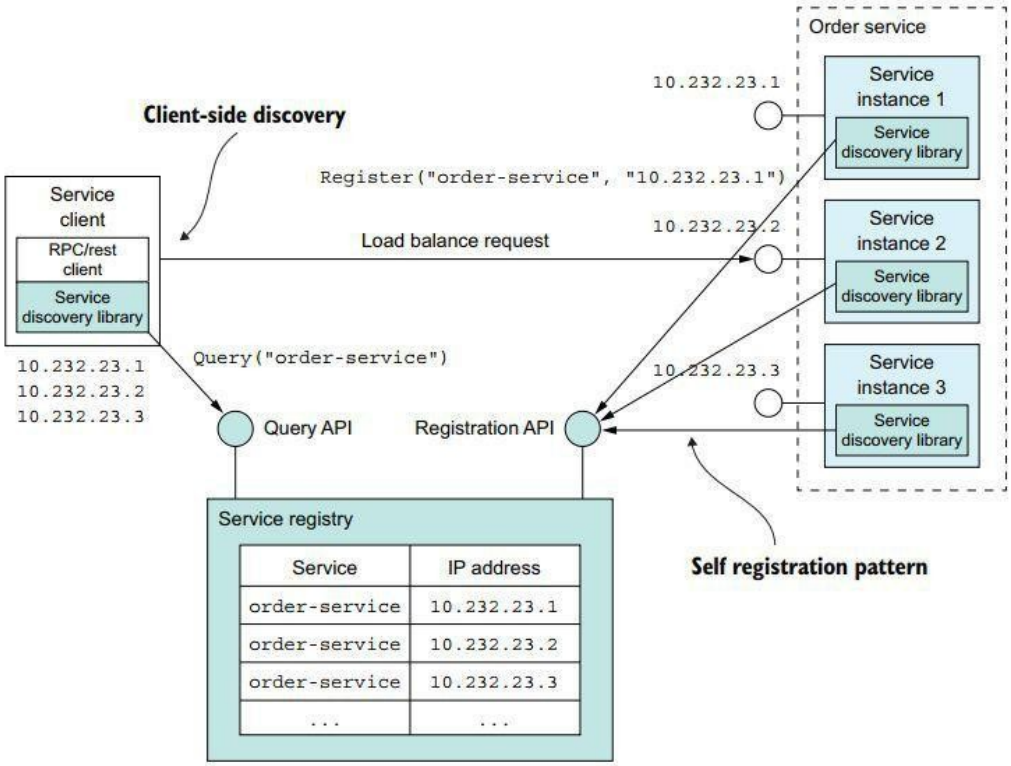


Rešenje bi bio neki vid automatizacije registrovanja lokacije servisa. Umesto da manuelno registrujemo lokacije servisa, oni će se sami (ili će ih neko drugi) javiti nekom centralnom registru koji će zabeležiti njihovu adresu i port. Taj registar je moguće kontaktirati i dobiti kompletan spisak svih pokrenutih instanci i zatim odrediti koju konkretnu instancu kontaktirati. Postoje dva načina da se implementira service discovery:

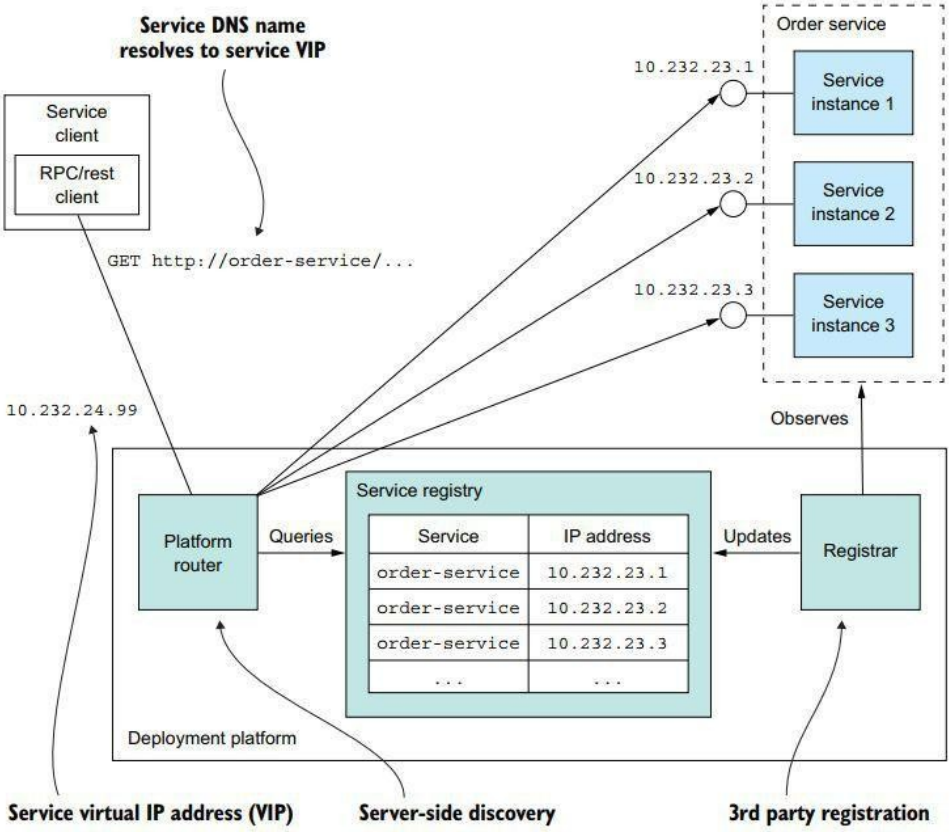
- Klijenti i servisi direktno interaguju sa registrom (Application-Level Service Discovery)
- Depolyment infrastruktura se brine za discovery (Platform-Provided Service Discovery)

Prvi način se sastoji u već opisanom toku. Dakle, instanca se javlja nekom centralnom registru, koji beleži lokaciju. Pored beleženja, stavka koja je takođe važna je Health-Checking, kako bi u slučaju otkaza neke instance bilo izbegnuto njeno kontaktiranje i samim tim izazivanje timeout-a ili dodatnog kašnjenja usled pronalaženja neke aktivne instance. Takođe, ovi sistemi se koriste i za potrebe distribuirane konfiguracije, što dodatno može olakšati održavanje infrastrukture. Kada klijent želi da se obrati nekoj instanci, kontaktira ”centralno čvorište” i dobija podatke na osnovu kojih može da pronađe tačan servis, odnosno konkretnu instancu upotrebom nekom load-balancing algoritma.

Prednosti ovakvog discovery mehanizma leže u tome da ne zavisimo od deployment platforme, ali nedostaci su ti da su često biblioteke usko vezane za neki jezik ili framework. Dodatno, opet u zavisnosti od tehnologija, često je potrebna konfiguracija samih servisa, a nekad sadrže i dodatan kod koji se odnosi na service discovery, što želimo da izbegnemo ako je moguće. Takođe, predstavljaju još jedan deo sistema koji treba održavati i za koji je neophodno obezbediti high-availability (izbeći SPOF - **S**ingle **P**oint **O**f **F**ailure).



Drugi oblik discovery-ja je pružen od strane deployment platforme kao što su Docker i Kubernetes koji su u stanju da u potpunosti odrade Service Discovery i routing što ne zahteva nikakve dodatne elemente koje treba održavati u arhitekturi. Ovo uključuje Server-side discovery (što nije nužno povezano sa deployment platformom).

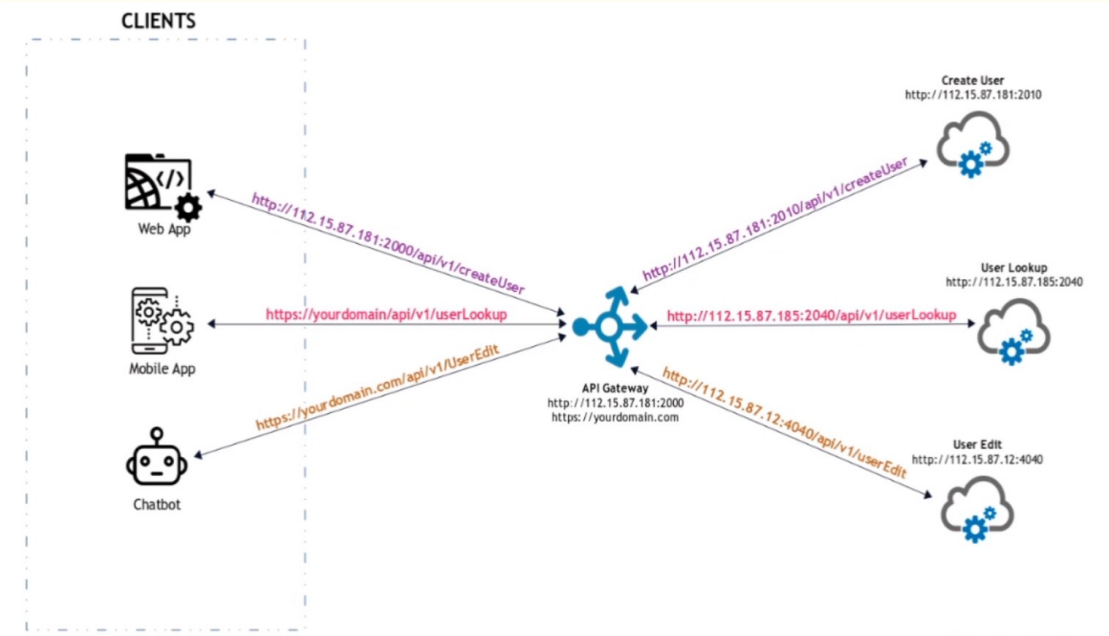


Umesto da instance direktno komuniciraju jedna sa drugom, zahtev se šalje ka ruteru, koji predstavlja ulaznu tačku, koji će se onda obratiti Service Registry-ju za neophodne informacije. Sa stanovišta registrovanja instanci one ne moraju same da se registruju, već ih može neko drugi registrovati (3rd party registration - registar).

Pored benefita u ovom obliku postoji i jedan očigledan nedostatak, a to je da je moguće vršiti discovery samo onih servisa koji su deploy-ovani upotrebom konkretne platforme.

3. API Gateway

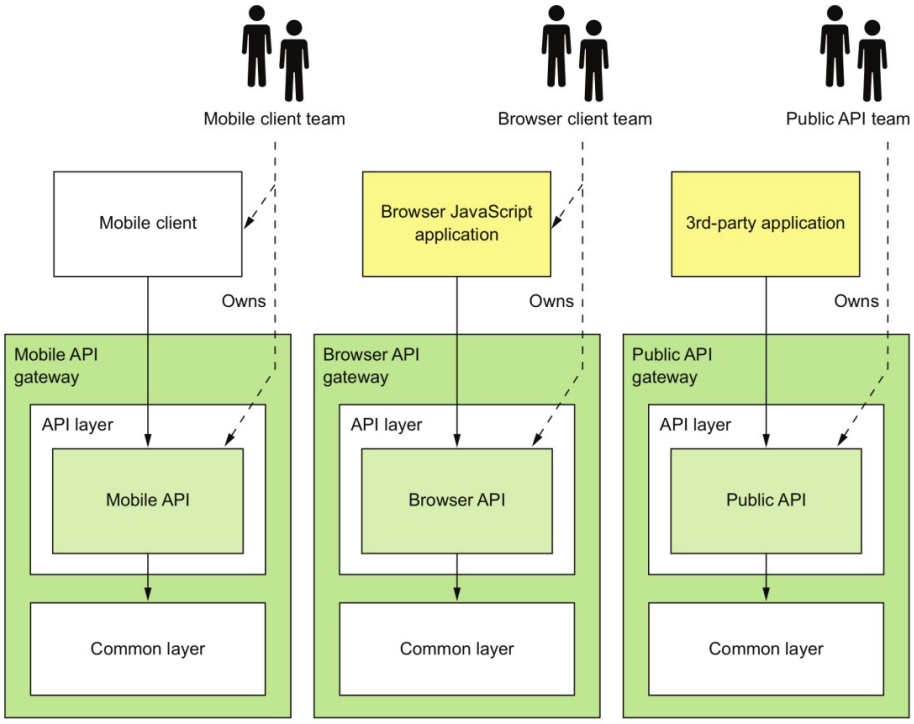
Mikroservisna arhitektura podrazumeva postojanje velikog broja međusobno nezavisnih servisa koji rade kao celina i opslužuju zahteve klijenata aplikacije. Neki od osnovnih problema sa takvom organizacijom su da klijent u sebi mora sadržati logiku za otkrivanje i adresiranje svakog od servisa koji treba da kontaktira, dalja dekompozicija servisa se komplikuje zbog toga što klijent direktno zavisi od unutrašnje organizacije sistema, protokoli koje servisi koriste nisu uvek pogodni itd. Kako bi se navedeni nedostaci prevazišli, uvodi se komponenta koja se naziva API Gateway. API Gateway je servis koji predstavlja jedinstvenu ulaznu tačku u sistem, enkapsulira unutrašnju arhitekturu i nudi API koji klijenti mogu koristiti.



Glavna zaduženja API Gateway-a su rutiranje pristiglih zahteva, API Composition (detaljnije o ovoj temi u sledećoj sekciji) i translacija protokola. Svi zahtevi koje eksterni klijenti šalju prvo stižu do API Gateway-a koji u zavisnosti od samog zahteva vrši prosleđivanje zahteva odgovarajućem servisu. Ukoliko zahtev podrazumeva dobavljanje podataka koje poseduju različiti servisi, API Gateway može uputiti odgovarajuće zahteve i agregirati

dobijene rezultate, a zatim ih proslediti klijentu. Što se tiče translacije protokola, API Gateway se često implementira tako da nudi RESTful API zbog rasprostranjenosti ovog arhitektonskog stila i raznovrsnosti klijenata koji ga podržavaju, dok aplikativni servisi interno mogu koristiti neke druge protokole, kao što je na primer gRPC. Kada postoje neslaganja u protokolima, API Gateway je zadužen da obavi neophodnu translaciju između eksternog i internog API-ja. Nije redak slučaj da API Gateway ima i dodatna zaduđenja, takozvane edge funkcionalnosti. Neke od njih su ograničenje stope pristupa, keširanje, prikupljanje metrika, logovanje.

Aplikacija može posedovati jedan API Gateway koji je zadužen za opsluživanje svih tipova klijenata. Potencijalni problem sa ovakvim pristupom je taj da različiti klijenti imaju različite potrebe i zahteve. Stoga, alternativni pristup je formirati API Gateway koji pruža svakom tipu klijenta odgovarajući API (Mobile API, Browser API, Public API itd). Na slici ispod prikazana je ilustracija Backend for frontend šablona koji podrazumeva postojanje odvojenog API Gateway-a za svaki tip klijenta.



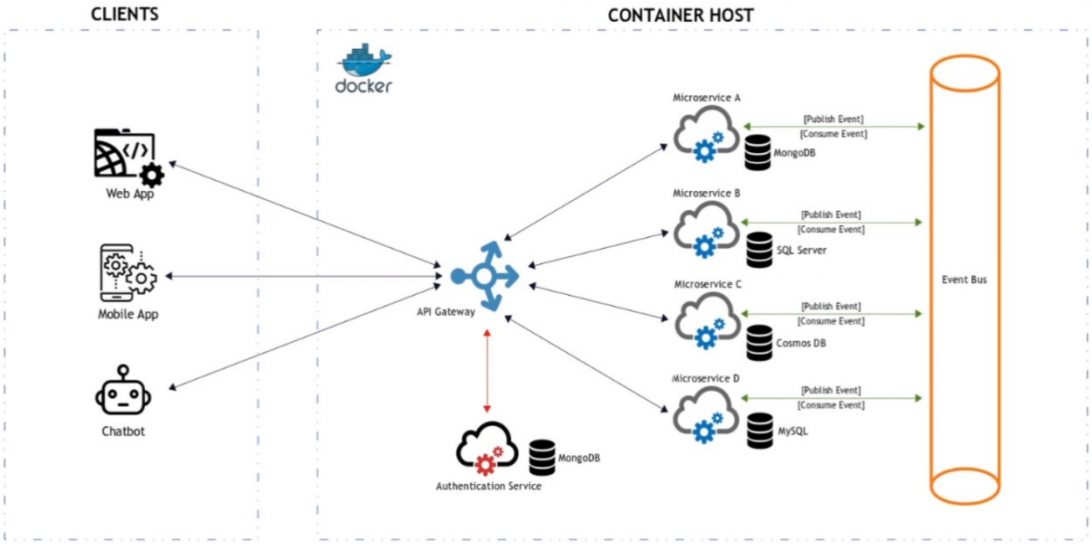
Uvođenjem API Gateway komponente postižu se benefiti kao što su enkapsulacija interne strukture aplikacije, pružanje API-ja specifičnih za različite klijente i pojednostavljenje klijentskog koda. Sa druge strane, uvode se neželjene pojave kao što su postojanje još jedne komponente koje se mora razvijati i održavati, a pritom mora biti visoko dostupna, neophodno je redovno ažurirati API Gateway kako bi API-ji koje servisi nude bili vidljvii spoljašnjem svetu. S obzirom da predstavlja ulaznu tačku, odnosno klijenti šalju zahteve na adresu koja je vezana za API Gateway i relativno je statička, skaliranje je drugačije prirode i pri lošoj konfiguraciji API Gateway može postati usko grlo

sistema. Kako predstavlja dodatnu tačka između klijenta i mikroservisa, uvodi se i dodatno kašnjenje u obradu svakog zahteva.

4. Tipična mikroservisna arhitektura

Komponente koje se mogu videti kao deo mikroservisne arhitekture variraju u zavisnosti od potreba sistema. Neke od osnovnih gradivnih jedinica koje se često sreću u rešenjima ovog tipa su sledeće:

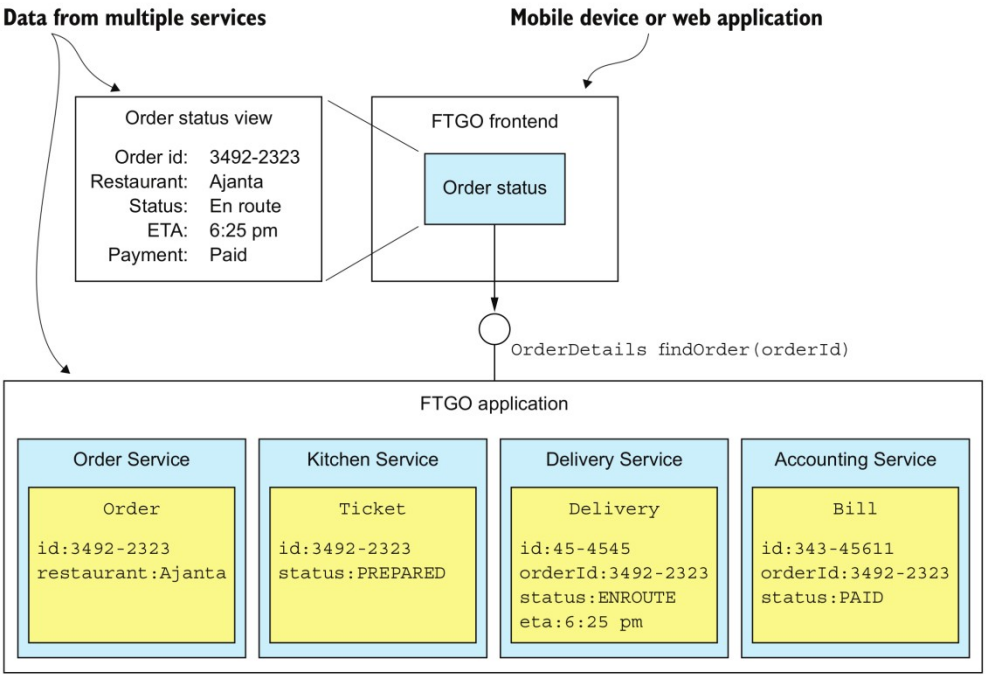
- Klijentske aplikacije - Prezentacioni sloj arhitekture.
- Mikroservisi - Svaki izvršava određenu biznis logiku i logiku za pristup podacima, neophodnu da se izvrši jedna funkcija ili proces. Mikroservisi međusobno ne dele podatke, odnosno svaki poseduje svoju bazu podataka. Čest je slučaj da servisi koriste različite tipove baza podataka, tako da oni budu odgovarajući datim slučajevima korišćenja.
- API Gateway - Predstavlja jedinstvenu ulaznu tačku ka mikroservisima. Umesto da klijent mora biti svestan svakog pojedinačnog servisa, on kontaktira API Gateway koji dalje prosleđuje zahtev.
- Authentication servis - Zadužen da autentifikuje klijentske aplikacije i obezbedi im token određenog životnog veka koji se šalje uz svaki zahtev kako bi se na siguran način pristupalo mikroservisima.
- Event Bus - Omogućava mikroservisima da komuniciraju asinhronim stilom baziranim na događajima (preferirani stil u odnosu na sinhronu komunikaciju, servisi su više loosely coupled), tako što svaki mikroservis može da objavljuje i/ili sluša događaje koje treba da obradi.



5. API Composition

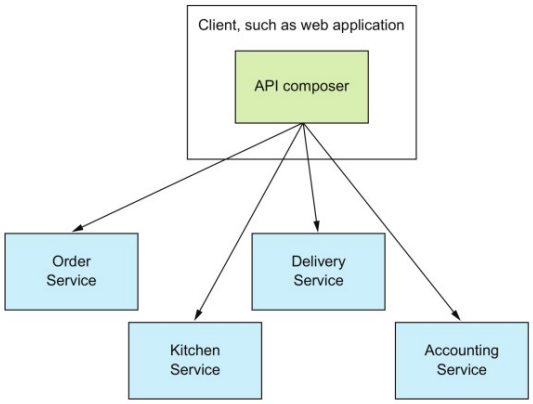
Formiranje upita u monolitnoj aplikaciji je često jednostavno zbog toga što ona poseduje jednu bazu podataka - svi neophodni podaci nalaze se na istom mestu. PIsanje upita u mikroservisnoj aplikaciji je izazovniji zadatak

zbog toga što se podaci često nalaze u različitim bazama podataka koje poseduju različiti servisi. Opisani problem ilustrovan je na slici ispod.



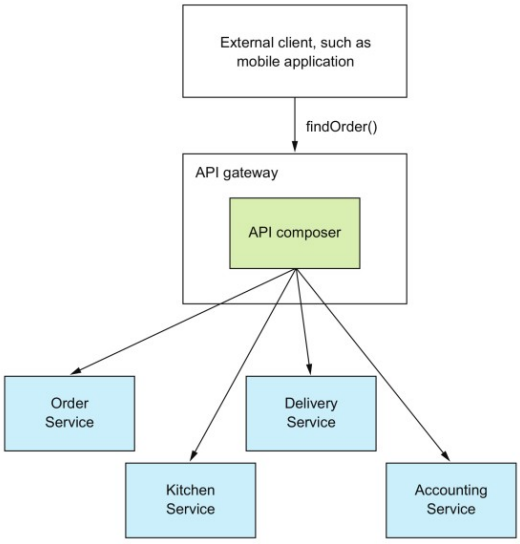
API Composition je jedan od šablona koji se može koristiti za rešavanje prikazanog problema. Suština je u tome da se upit formira tako da se iz jedne tačke iniciraju svi zahtevi koji dobavljaju neophodne podatke, a da se zatim na istom tom mestu rezultati agregiraju u jednu celinu. Komponenta koja obavlja navedeni proces naziva se *API Composer*, dok se servisi iz kojih se dobavljaju podaci nazivaju *provider services*.

Kada se koristi navedeni šablon, postavlja se pitanje koja komponenta u sistemu treba da preuzme ulogu API Composer-a.

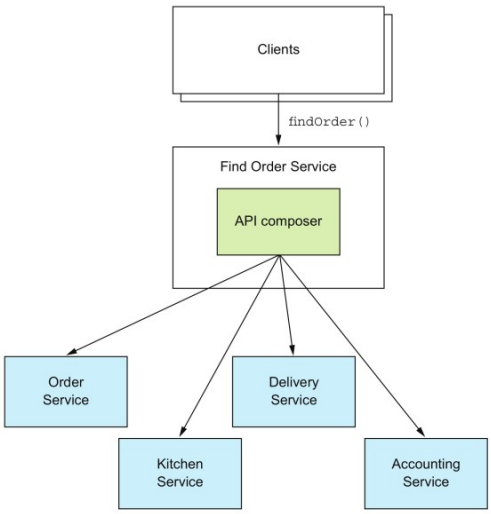


Prva opcija je da se neophodna logika ugradi u klijenta aplikacije. Ovakvo rešenje zahteva od klijenta da kontaktira svaki servis, a zatim agregira dobijene rezultate.

Druga opcija je da API Gateway bude zadužen za obavljanje kompozicije. Ovakva organizacija ima smisla u situaciji kada je rezultat API Composition-a deo eksternog API-ja sistema i koriste ga klijenti. Dodatni benefit je da se pristigli zahtev ne mora dalje rutirati nekom drugom



servisu, a klijent jednim upućenim zahtevom dobija agregiran rezultat, što nije slučaj kada se implementira prva navedena opcija.



Treća mogućnost je da se API Composer razvije kao novi servis kom je to primarna uloga. Ovu strategiju pogodno je koristiti kada se rezultat upita koristi interno među ostalim servisima i kada je logika agregiranja previše kompleksna da bi bila deo API Gateway

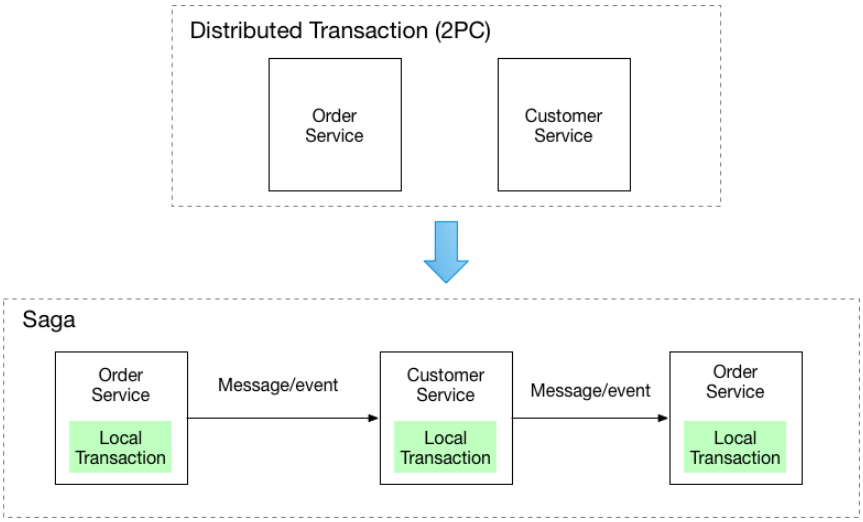
API Composition je jednostavan šablon koji se može primeniti u velikom broju slučajeva kada se rezultat upita formira dobavljanjem podataka koje poseduje više od jednog mikroservisa. Međutim, postoje situacije kada je spajanje podataka u memoriji veoma skupo ili nemoguće zbog veličine tih podataka, kada je bolje primeniti CQRS šablon. Ostali nedostaci uključuju dodatno trošenje računarskih i mrežnih resursa jer je potrebno izvršiti više zahteva i više upita ka bazi, rizik od smanjene dostupnost zbog toga što barem tri komponente učestvuju u čitavom procesu, kao i nekonzistentnost u podacima izazvanu time što različiti servisi mogu posedovati različite verzije podataka u određenom trenutku.

6. Sage

Upotreba mikroservisne arhitekture uvodi dodatnu kompleksnost u mnoge aspekte razvoja, deploymena i testiranja aplikacije. Jednan od problema koji do sada nije pokriven je kako se nositi sa transakcijama. Transakcije koje se izvršavaju u okviru jednog mikroservisa i dalje jednostavno mogu zadovoljiti ACID (atomicity, consistency, isolation, durability) svojstva, dok se problem javlja kada je potrebno izmeniti podatke u više od jednog mikroservisa, gde svaki od njih poseduje odvojenu bazu podataka.

Protokoli za upravljanje distribuiranim transakcijama, kao što su 2PC (Two-phase commit), bave se navedenim problemom i obezbeđuju da se izmene izvrše ili kod svih učesnika transakcije ili ni kod jednog, ostavljajući sistem u konzistentnom stanju. 2PC funkcioniše tako što podrazumeva postojanje koordinatora koji u prvoj fazi (faza pripreme i glasanja) pita svakog učesnika da li je spreman da izvrši commit nakon čega se šalje da/ne odgovor, a zatim se u drugoj fazi (commit faza) vrši commit ukoliko su svi

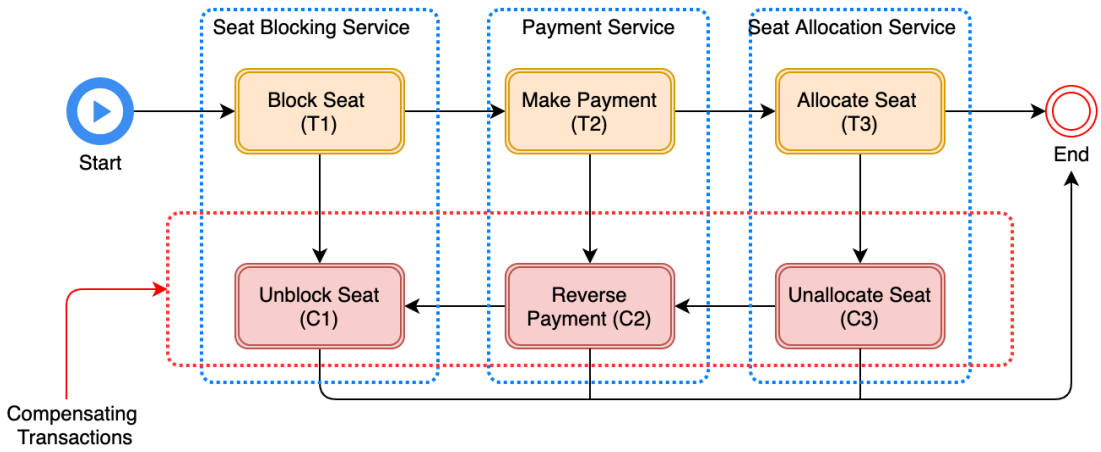
učesnici pozitivno glasali ili rollback ukoliko je bar jedan od njih glasao negativno. Uprkos problemu koji rešava, 2PC se ne preporučuje u određenim situacijama zbog svoje sinhrone prirode. 2PC je blokirajući protokol i poseduje SIngle Point of Failure, što je koordinator, koji prilikom otkaza može ostaviti učesnike u nekonzistentnom stanju. Dodatan problem je kašnjenje jer će čitav sistem biti spor koliko i najsporija transakcija u čitavom procesu.



Alternativni mehanizam koji se može koristiti za navedeni problem je saga. Saga predstavlja sekvencu lokalnih transakcija, takvih da svaka lokalna transakcija vrši atomičnu izmenu podataka u okviru jednog servisa. Zadatak sage je da obezbedi konzistentnost podataka u mikroservisnoj arhitekturi. Za svaku transakciju koja obuhvata više mikroservisa definiše se po jedna saga. Treba napomenuti da sage same po sebi ne garantuju svojstvo izolovanosti, već se ono mora obezbediti u aplikativnom sloju.

Kada određeni signal inicira sagu, počinje da se izvršava prva lokalna transakcija i po njenom završetku okida se naredna lokalna transakcija. Sage se izvršavaju asinhronom komunikacijom među učesnicima, čime se osigurava da će svaki korak biti izvršen čak i kada je neki od učesnika privremeno nedostupan.

Kako tokom trajanja sage svaka lokalna transakcija kominutuje izmene, potrebno je obezbediti rollback mehanizam koji se može pokrenuti kada neka od transakcija nije u stanju da se izvrši jer uslovi za to nisu ispunjeni. Rollback prethodno izvršenih transakcija vrši se compensating transakcijama. Ukoliko je uspešno izvršeno prvih n transakcija, a prva sledeća je bila neupešna, potrebno je izvršiti takođe n compensating transakcija koje će se obaviti u obrnutom redosledu u odnosu na originalne transakcije koje su načinile inicijalnu izmenu. Kako saga može sadržati read-only korake, nije nužno da svaki korak poseduje svoju compensating transakciju.



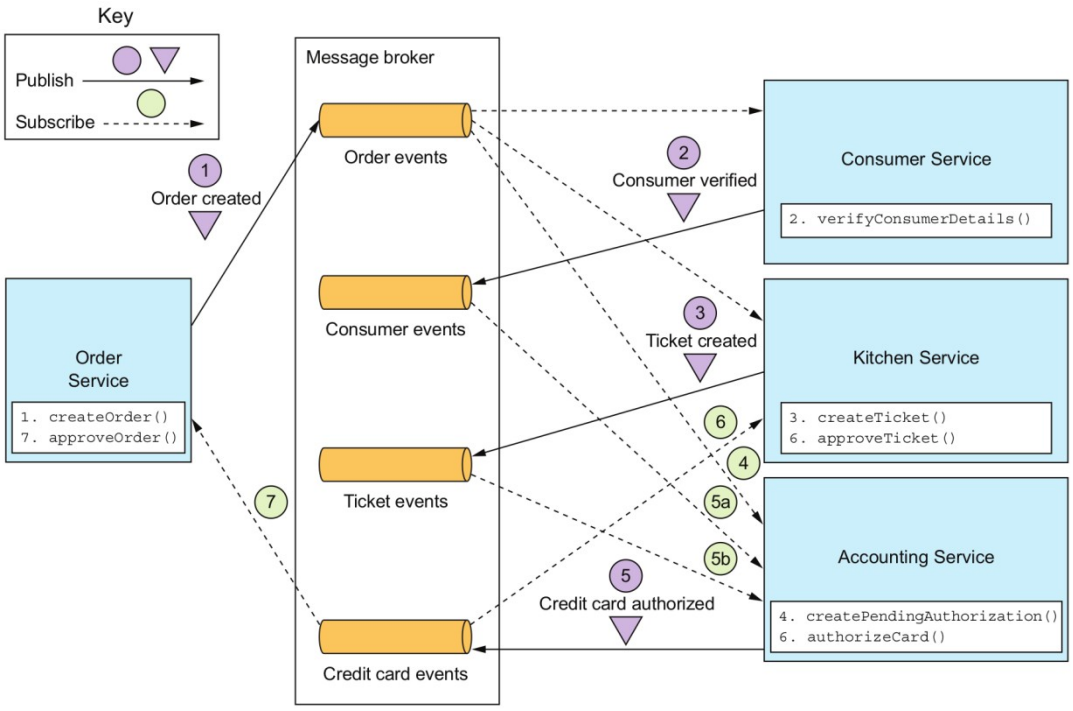
Osnovno pitanje pri formiranju sage je njeno koordinisanje, odnosno ko je zadužen da obavesti učesnika sage da treba da izvrši svoju lokalnu transakciju. Donošenje neophodnih odluka može biti decentralizovano (koreografija) ili centralizovano (orkestracija).

6.1. Koreografija

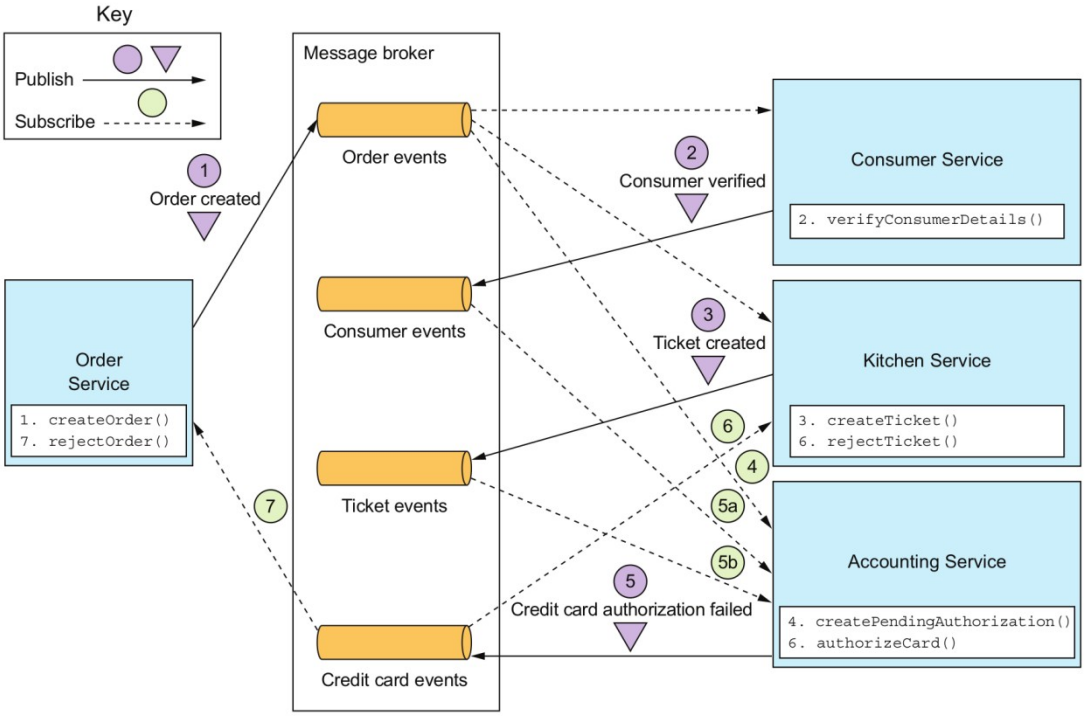
Kada se koordinisanje sage vrši koreografijom, ne postoji centralni koordinator koji je zadužen da obavesti učesnike šta treba da rade, već se oni prijave za slušanje događaja koje je poslao neki drugi učesnik, a koji su njima od interesa i zahtevaju njihovu reakciju.

Na slici ispod prikazan je primer sage i tok transakcija koje učesnici izvršavaju. U slučaju happy path-a koraci su sledeći:

- Order Service kreira Order koji je u *ApprovalPending* stanju i objavljuje OrderCreated događaj
- Consumer Service registruje OrderCreated događaj, verifikuje da li ta mušterija može da izvrši narudžbinu i objavljuje ConsumerVerified događaj
- Kitchen Service registruje OrderCreated događaj, validira Order i kreira Ticket u *CreatePending* stanju, a zatim šalje TicketCreated događaj
- Accounting Service reaguje na OrderCreated događaj i kreira CreditCardAuthorization u *Pending* stanju
- Accounting Service registruje TicketCreated i ConsumerVerified događaje, menja stanje na korisnikovoj kreditnoj kartici i objavljuje CreditCardAuthorized događaj
- Kitchen Service registruje CreditCardAuthorized događaj i menja stanje Ticket-a u *AwaitingAcceptance*
- Order Service prima CreditCardAuthorized poruku, menja stanje Order-a u *Approved* i objavljuje OrderApproved događaj



Napomena je da ovakva sekvenca događaja nije jedini tok i da se u bilo kom koraku može desiti neuspešna transakcija koja treba da pokrene sve odgovarajuće compensating transakcije. Na slici ispod dat je primer alternativnog neuspšnog toka gde sredstva sa kartice nisu mogla biti skinuta.



Koraci koji su izvršeni u okviru sage su sledeći:

- Order Service kreira Order koji je u *ApprovalPending* stanju i objavljuje OrderCreated događaj

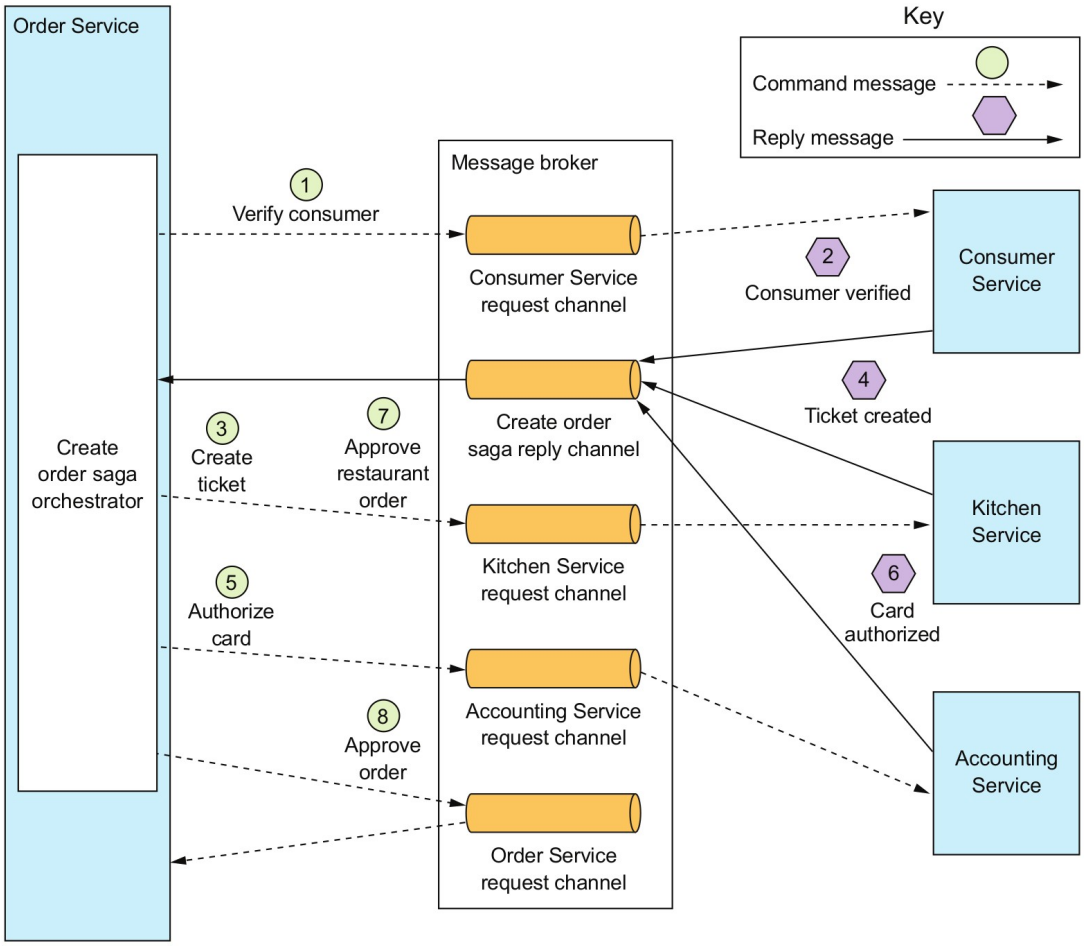
- Consumer Service registruje OrderCreated događaj, verifikuje da li ta mušterija može da izvrši narudžbinu i objavljuje ConsumerVerified događaj
- Kitchen Service registruje OrderCreated događaj, validira Order i kreira Ticket u *CreatePending* stanju, a zatim šalje TicketCreated događaj
- Accounting Service reaguje na OrderCreated događaj i kreira CreditCardAuthorization u *Pending* stanju
- Accounting Service registruje TicketCreated i ConsumerVerified događaje, pokušava da izmeni stanje na korisnikovoj kreditnoj kartici i objavljuje CreditCardAuthorizationFailed događaj
- Kitchen Service registruje CreditCardAuthorizationFailed događaj i menja stanje Ticket-a u *Rejected*
- Order Service prima CreditCardAuthorizationFailed poruku i menja stanje Order-a u Rejected

Kako bi saga bila uspešno izvršena, neophodno je obezbediti pouzdanu komunikaciju baziranu na događajima. Prvi problem je taj da izmena podataka u bazi i objavljivanje odgovarajućeg događaja treba da budu atomični, odnosno treba se obezbediti da ako se izvrši prva akcija, i druga uvek mora biti izvršena. Drugi problem je mapiranje događaja koji se objavljuju na podatke koje servisi poseduju. U slučaju gore navedenog primera, identifikator narudžbine može biti ta informacija koja omogućava mapiranje.

Sage koje koriste koreografiju su pogodne pri jednostavnim slučajevima. Međutim, kako one postaju kompleksnije, razumevanje je otežano zbog toga što je donošenje odluka raspoređeno u više servisa i u takvim situacijama bolje je koristiti orkestraciju koja je centralizovana.

6.2. Orkestracija

Kada se saga koordiniše orkestracijom, potrebno je formirati orkestrator komponentu čije je jedino zaduženje da govori učesnicima sage šta da rade izdajući im instrukcije. Komunikacija između orkestratora i učesnika je bazirana na command async reply stilu komunikacije. Kako bi se jedan korak sage izvršio, orkestrator pošalje komandu odgovarajućem učesniku i kada on izvrši tu operaciju, pošalje odgovor orkestratoru koji govori o tome kakav je ishod akcije bio. Orkestrator zatim obrađuje odgovor i utvrđuje koji korak je sledeće neophodno izvršiti.



Primer dat na slici iznad izvršava istu funkcionalnost kao i primer koji koristi koreografiju. Orkestrator je u ovom slučaju deo Order Service-a, ali komande se tom servisu šalju na isti način kao i ostalim servisima radi konzistentnosti, odnosno svejedno mu je ko je učesnik. Koraci u slučaju happy path-a su sledeći:

- Orkestrator šalje VerifyConsumer komandu Consumer Service-u
- Consumer Service odgovara ConsumerVerified porukom
- Orkestrator šalje CreateTicket komandu Kitchen Service-u
- Kitchen Service odgovara TicketCreated porukom
- Orkestrator šalje AuthorizeCard komandu Accounting Service-u
- Accounting Service odgovara CardAuthorized porukom
- Orkestrator šalje ApproveTicket komandu Kitchen Service-u
- Orkestrator šalje ApproveOrder komandu Order Service-u

Napomena je da iako je prikazan samo jedan scenario, koji prikazuje uspešno kreiranje narudžbine, pored njega može postojati i više neuspešnih scenarija, npr. kada mušterija nije verifikovana ili kartica nije autorizovana.

Iako je orkestracija pogodna za kompleksne scenarije i centralizovan orkestrator enkapsulira logiku koordinacije, mora se voditi računa da se u njega ne uključi previše biznis logike, već da bude zadužen samo za izdavanje komandi i procesiranje odgovora.

7. Dodatni materijali

- Using sagas to maintain data consistency, Chris Richardson
- Microservices patterns, Chris Richardson
- Building Microservices, Sam Newman
- Saga Pattern examples
- Sagas, Hector Garcia-Molina, Kenneth Salem